ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN



MÔN HỌC: CSC14003 - Cơ Sở Trí Tuệ Nhân Tạo

BÁO CÁO ĐỒ ÁN MÔN HỌC

Project 1: Bloxorz Solver

Giảng viên hướng dẫn: TS. Bùi Tiến Lên

TA. Nguyễn Thanh Tình

Lớp: CQ2024/22

Năm học: Học kỳ I, 2026 - 2027

Sinh viên thực hiện:

Ngô Hoàng Minh - 24120381

Nguyễn Xuân Lộc - 24120369

Đỗ Ngọc Hải - 24120300

Nguyễn Thành Bảo - 24120023

Phan Nhật Minh - 24120384

Nội dung

1 Giới thiệu **1**

1.1 Thông tin nhóm 1

2 Project Planning & Task Distribution **1**

2.1 Kế hoạch triển khai dự án 1

2.2 Phân công nhiệm vụ 3

3 Thiết kế BackEnd **3**

3.1 Tổ chức và mô hình trạng thái 3

3.2 Thiết kế hướng đối tượng 4

3.3 Các hàm cơ sở 4

3.4 Luồng xử lý hành động 6

4 Thiết kế FrontEnd **6**

4.1 Tổ chức module và luồng dữ liệu 6

4.2 Thiết kế các lớp chính 7

4.3 Các cơ chế dựng hình và hoạt ảnh 8

4.4 Luồng xử lý và tiến trình chơi 9

5 Tìm kiếm theo chiều sâu (DFS) **9**

5.1 Mô hình trạng thái và chi phí 9

5.2 Cấu trúc dữ liệu và quá trình tìm kiếm 10

5.3 Tính chất và giới hạn 10

5.4 Giao diện và kiểm thử 10

6 Tìm kiếm theo chi phí đồng nhất (UCS) **12**

6.1 Mô hình trạng thái và chi phí 12

6.2 Cấu trúc dữ liệu và quá trình tìm kiếm 12
6.3 Tính chất và giới hạn 14

7 Thuật toán A* 14

7.1 Mô hình hoá bài toán 14
7.2 Về hàm Heuristic 17
7.2.1 Xây dựng bài toán nới lỏng 17
7.2.2 Xây dựng bảng khoảng cách bằng BFS ngược 18
7.2.3 Admissible and Consitent 18
7.2.4 Chi phí tính toán và giới hạn 19

8 Tích hợp thuật toán vào giao diện game 20

8.1 Phạm vi tích hợp 20
8.2 Chạy, dừng và phát lại lời giải 20
8.3 Hiển thị kết quả và xuất dữ liệu 21
8.4 Tương thích sau khi tích hợp vào main 21

9 Benchmark và trực quan hóa kết quả 21

9.1 Thiết kế phép đo 21
9.2 Schema CSV và kiểm chứng dữ liệu 22
9.3 Kết quả trên 20 maps 23
9.4 Lựa chọn biểu đồ 24
9.5 Hạn chế và hướng mở rộng 24

1. Giới thiệu

1.1. Thông tin nhóm

Nhóm em gồm có 05 thành viên, với thông tin và mức độ hoàn thành như sau:

Bảng 1: Thành viên nhóm và mức độ hoàn thành

Họ tên	MSSV	Mức độ hoàn thành
Ngô Hoàng Minh	24120381	100%
Nguyễn Xuân Lộc	24120369	100%
Đỗ Ngọc Hải	24120300	100%
Nguyễn Thành Bảo	24120023	100%
Phan Nhật Minh	24120384	100%

2. Project Planning & Task Distribution

2.1. Kế hoạch triển khai dự án

Dự án Bloxorz Solver được chia thành các nhóm công việc chính: xây dựng logic trò chơi, phát triển giao diện, cài đặt các thuật toán tìm kiếm và chuẩn bị báo cáo. Logic trò chơi và mô hình trạng thái là nền tảng dùng chung, giúp các thành viên phát triển giao diện và thuật toán song song mà vẫn sử dụng thống nhất các quy tắc di chuyển, công tắc và cầu.

Kế hoạch triển khai dưới đây được tổng hợp từ lịch sử commit trong khoảng ngày 12/10/2026 đến 22/10/2026. Các giai đoạn có thời gian giao nhau do nhiều mô-đun được phát triển đồng thời.

Bảng 2: Các giai đoạn triển khai dự án

Giai đoạn	Thời gian	Nội dung chính
Xây dựng nền tảng	12–15/10	Khởi tạo repository và cấu trúc mã nguồn; xây dựng các đối tượng trò chơi, `Board`, `GameRules`, dữ liệu 20 màn chơi và các kiểm thử logic.
Phát triển giao diện	17–19/10	Xây dựng ứng dụng Pygame, bộ điều khiển và hiển thị bàn chơi; bổ sung menu, quản lý tiến trình, hiệu ứng chuyển động và sửa lỗi tương tác.
Phát triển thuật toán	19–22/10	Mô hình hóa bài toán; xây dựng khung solver dùng chung; cài đặt BFS, DFS, UCS và A*; bổ sung heuristic, mã đo hiệu năng và kiểm thử thuật toán.
Viết báo cáo	Từ ngày 19	Viết báo cáo LaTeX, chuẩn bị video

2.2. Phân công nhiệm vụ

Bảng 3: Phân công nhiệm vụ của các thành viên

Thành viên	Nhiệm vụ
Ngô Hoàng Minh	• Khởi tạo dự án, tổ chức mã nguồn và dữ liệu dùng chung. • Xây dựng `Board`, `GameRules`, bộ 20 màn chơi và công cụ trực quan hóa bản đồ. • Mô hình hóa bài toán, xây dựng khung solver, cài đặt A* và heuristic. • Viết kiểm thử logic, bản đồ và A*; sửa lỗi GUI.
Phan Nhật Minh	• Định nghĩa các loại ô và cầu; khai báo thư viện phụ thuộc. • Cài đặt thuật toán BFS. • Bổ sung mã đo thời gian, bộ nhớ, số nút mở rộng và độ dài lời giải.
Nguyễn Thành Bảo	• Xây dựng cơ chế khối, công tắc và tương tác với cầu. • Tích hợp các thành phần vào API backend dùng chung. • Cài đặt UCS, viết kiểm thử và tài liệu thuật toán.
Đỗ Ngọc Hải	• Xây dựng ứng dụng Pygame, bộ điều khiển và hiển thị trò chơi. • Quản lý tiến trình, sửa hiệu ứng rơi và viết kiểm thử giao diện, chuyển động. • Cài đặt DFS, bổ sung cấu trúc kết quả tìm kiếm và kiểm thử DFS.
Nguyễn Xuân Lộc	• Điều chỉnh tỷ lệ hiển thị bàn chơi, khối và ký hiệu công tắc. • Hoàn thiện menu, màn hình chiến thắng và hiệu ứng ngôi sao. • Cải thiện hiệu ứng rơi và bổ sung cơ chế tự khởi động lại màn chơi khi thua.

3. Thiết kế BackEnd

3.1. Tổ chức và mô hình trạng thái

BackEnd được tổ thiết kế thành 3 thành phần: các đối tượng trong `BackEnd/objects`, bộ luật trong game `BackEnd/logic`, các thuật toán tìm kiếm trong `BackEnd/Solver`. Giao diện và các solver sử dụng chung bộ luật để đảm bảo tính nhất quán.

Lớp `Board` quản lí các ô, công tắc và trạng thái hiện tại. Trạng thái tìm kiếm

được biểu diễn bởi

$$s = (P, o, a, B)$$

trong đó P là các tọa độ khối chiếm giữ, o là hướng đặt hoặc trạng thái tách, a là khối con đang điều khiển và B là tập cầu đang mở.

`BlockState` lưu P, o, a; `GameState` kết hợp `BlockState` với B. Hai lớp được khai báo bất biến, cho phép dùng trực tiếp làm khóa trong `visited` hoặc bảng chi phí. Tọa độ sử dụng dạng (x, y), với x tăng sang phải và y tăng xuống dưới.

3.2. Thiết kế hướng đối tượng

Lớp `Tile` là lớp cơ sở của các loại ô. Các lớp con định nghĩa khả năng đỡ khối thông qua `can_support()`:

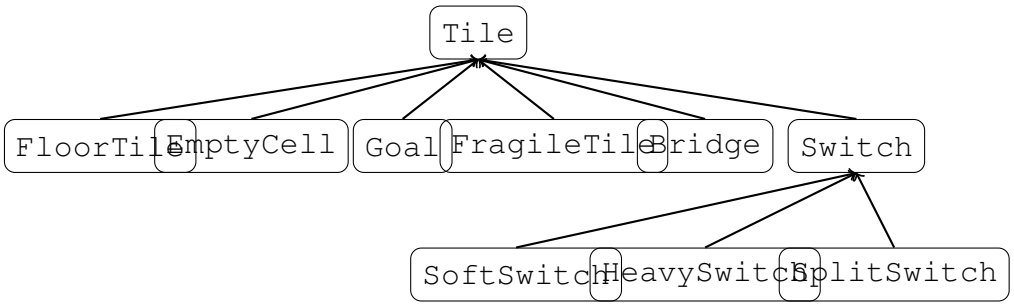
- `FloorTile`, `Goal`: cho phép khối nằm trên ô; điều kiện chiến thắng được kiểm tra riêng.
- `EmptyCell`: không đỡ khối. Vị trí không có đối tượng ô trên bàn cũng được xem là khoảng trống.
- `FragileTile`: không đỡ khối nguyên vẹn đang đứng; cho phép khối nằm hoặc khối con sau khi tách.
- `Bridge`: chỉ đỡ khối khi ID cầu thuộc tập `open_bridges` của trạng thái hiện tại.

Lớp `Switch` kế thừa `Tile`. `SoftSwitch` kích hoạt khi có phần khối đè lên; `HeavySwitch` và `SplitSwitch` chỉ kích hoạt khi khối nguyên vẹn đứng trên công tắc. Các liên kết `SwitchBridgeLink` xác định cầu bị tác động theo chế độ đảo trạng thái (`TOGGLE`) hoặc đặt trạng thái (`SET`). `SplitSwitch` còn lưu hai vị trí dịch chuyển của các khối con.

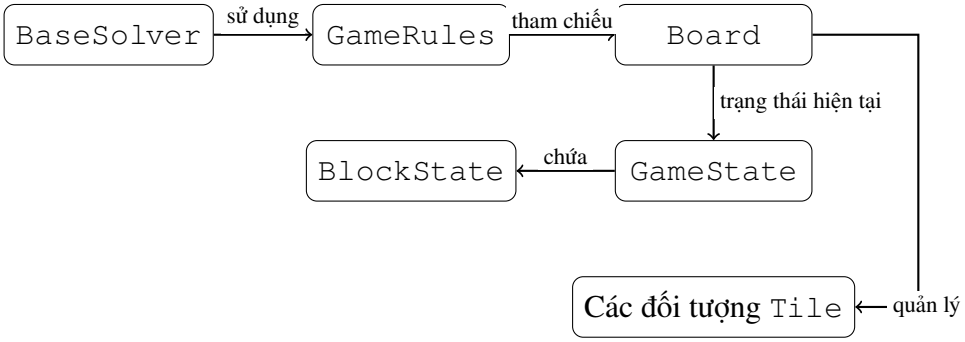
Các lớp `BFSSolver`, `DFSSolver`, `UCSSolver` và `AStarSolver` kế thừa `BaseSolver`, dùng chung cách sinh trạng thái và truy vết lời giải.

3.3. Các hàm cơ sở

`init_map(map_name)` Đọc dữ liệu JSON, tạo các đối tượng trên bàn và khởi tạo trạng thái bắt đầu.



Hình 1: Quan hệ kế thừa giữa các loại ô. Mũi tên hướng từ lớp con đến lớp cha.



Hình 2: Quan hệ sử dụng và chứa giữa các lớp chính, không biểu diễn quan hệ kế thừa.

`Board.initialize_game_state()` Khởi tạo trạng thái khối và tập cầu mở theo cấu hình `initial_open`.

`Board.get_tile(position)` Tra cứu ô tại tọa độ, ưu tiên cầu phủ lên ô nền; trả `None` nếu không có ô.

`Board.add_game_state(state)` Cập nhật trạng thái hiện tại và đồng bộ thuộc tính khối của bàn chơi.

`GameRules.is_valid(state)` Kiểm tra hình dạng khối, cấu hình cầu và khả năng đỡ khối tại mọi vị trí đang chiếm giữ.

`GameRules.apply_action(state, action)` Tính trạng thái kế tiếp sau một hành động. Hàm xử lý di chuyển, công tắc, cầu và tách/ghép khối; trả `None` nếu không hợp lệ, không sửa bàn chơi hay trạng thái đầu vào.

`GameRules.is_goal(state)` Kiểm tra trạng thái hợp lệ và khối nguyên vẹn đứng thẳng trên ô đích.

`BaseSolver.successors(state)` Sinh các bộ $(action, next_state, cost)$ hợp lệ theo thứ tự hành động chung, với chi phí mỗi hành động bằng 1.

`BaseSolver.reconstruct_path(goal_node)` Lần theo `parent`, thu thập `action` và đảo thứ tự để trả chuỗi hành động từ đầu đến đích.

3.4. Luồng xử lý hành động

Với hành động di chuyển, `apply_action()` tính tư thế và vị trí mới, kiểm tra khả năng đứng trên các ô, áp dụng công tắc rồi kiểm tra lại trạng thái sau khi hành động. Nếu khối đang tách, hai khối con sẽ được hợp nhất khi nằm kề nhau. Hành động `Switch` chỉ đổi khối đang điều khiển và chỉ hợp lệ khi khối đang được tách.

Giao diện cập nhật bàn chơi bằng trạng thái trả về, các solver dùng trạng thái đó để mở rộng tìm kiếm. Do đó chơi thủ công và phát lại lời giải đều tuân theo cùng một bộ luật.

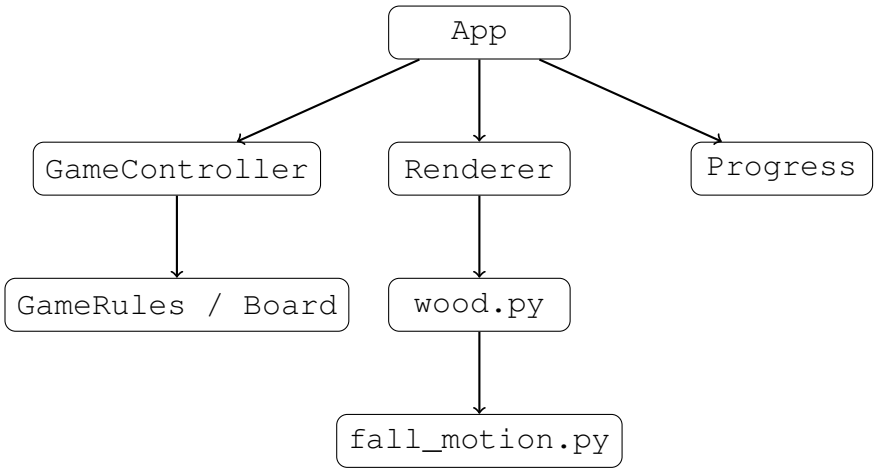
4. Thiết kế FrontEnd

4.1. Tổ chức module và luồng dữ liệu

FrontEnd được xây dựng bằng *Pygame*, tách thành các module theo vai trò riêng biệt trong `FrontEnd/`:

- `app.py` - lớp `App`, điều phối toàn bộ vòng lặp giao diện: xử lý sự kiện, vẽ khung hình, chuyển chế độ chơi và quản lý tiến trình tìm kiếm.
- `controller.py` - lớp `GameController`, cầu nối giữa giao diện và `BackEnd`.
- `renderer.py` - dựng hình bàn chơi theo phép chiếu isometric.
- `wood.py` - dựng hình khối gỗ 3D và hoạt ảnh lăn/rơi.
- `fall_motion.py` - tính tư thế rơi của khối khi thua.
- `progress.py` - lưu tiến trình chơi, tính sao và điều kiện mở khóa màn.

Dữ liệu đi theo một chiều: `BackEnd` sinh `GameState` mới sau mỗi hành động, `GameController` lưu lại trạng thái đó, còn `Renderer/wood.py` chỉ đọc trạng thái và tham số hoạt ảnh để vẽ, không module FrontEnd nào chỉnh sửa trực tiếp `Board` hay `GameRules` ngoài thông qua `GameController`. Nhờ vậy chơi thủ công, hiển thị lại lời giải và vẽ lại khung hình khi đổi kích thước cửa sổ đều dùng chung một nguồn trạng thái duy nhất.



Hình 3: Quan hệ sử dụng giữa các module của FrontEnd.

4.2. Thiết kế các lớp chính

Các lớp của FrontEnd được tổ chức theo quan hệ *composition*: mỗi lớp chỉ giữ tham chiếu tới đối tượng cần dùng chứ không sao chép trạng thái, tránh lệch dữ liệu giữa giao diện và bộ luật.

- `GameController` đóng gói một `Board` và `GameRules` cho một màn chơi, đồng thời lưu `history` (các cặp trạng thái-hành động đã thực hiện) phục vụ hiển thị số bước và chơi lại màn.
- Hàm `move(action)` gọi `GameRules.apply_action()`; nếu kết quả là `None` thì trạng thái được đánh dấu `"lost"`, ngược lại trạng thái mới được ghi vào `Board` và kiểm tra đích bằng `GameRules.is_goal()`. Vì chơi thủ công và phát lại lời giải đều đi qua cùng hàm này, hành vi luôn khớp với bộ luật dùng trong tìm kiếm.
- Tham số `replay` phân biệt hai luồng gọi: khi `replay=False` (người chơi tự di chuyển), lời giải và kết quả benchmark cũ bị xóa qua `clear_solution()` vì chúng không còn khớp với trạng thái hiện tại; khi `replay=True`, lịch sử tìm kiếm được giữ nguyên.
- Các hàm `set_solution()`, `start_replay()` và `step()` phục vụ riêng cho luồng phát lại lời giải, được mô tả chi tiết ở Mục 8.
- `Renderer` không giữ trạng thái trò chơi: mọi lời gọi vẽ nhận `state` và tham số hoạt ảnh làm đầu vào, nên cùng một lớp có thể vẽ cả khi chơi thủ công, khi phát lại và khi chỉ xem lại một khung hình tĩnh.

- `Progress` tách biệt hoàn toàn khỏi logic trò chơi, chỉ nhận tên màn và số bước để tính sao và lưu tiến độ xuống đĩa, nên không phụ thuộc `GameRules` và không bị ảnh hưởng nếu luật chơi thay đổi.

4.3. Các cơ chế dựng hình và hoạt ảnh

Phép chiếu isometric `Renderer.board()` chiếu tọa độ lưới (x, y, z) sang tọa độ màn hình bằng

$$(u, v) = \big(o_x + (x - y) \cdot s,\ o_y + (x + y) \cdot s/2 - z \cdot s\big),$$

với tỉ lệ s tính để bàn chơi luôn vừa khung hiển thị và gốc chiếu (o_x, o_y) đặt tại tâm khung.

Vẽ ô (`prism()`) Mỗi ô được vẽ bằng một hình hộp thấp, tô màu theo loại ô: nền, ô dễ vỡ (cam), cầu (nâu, gạch chéo đỏ khi đóng) và công tắc (vòng tròn `O`/`X`/hai chấm tương ứng `SoftSwitch`/`HeavySwitch`/`SplitSwitch`). Cầu đang đóng được bỏ qua khi vẽ nhưng vẫn vẽ ô nền dưới cầu, vì `Board` cho phép cầu phủ lên một ô khác; trạng thái cầu dùng để vẽ là trạng thái *trước* khi hoạt ảnh lăn bắt đầu, để cầu không đổi trạng thái giữa lúc khối đang lăn qua.

Hoạt ảnh lăn (`wood.py`) Khối xoay quanh cạnh tiếp đất theo hướng di chuyển (tham số `pivot`), với góc xoay $\theta(t) = \text{smoothstep}(t) \cdot \pi/2$, $t \in [0, 1]$ theo thời gian lăn (tối đa 0.28s hoặc theo tốc độ phát lại), trong đó *smoothstep* là $t^2(3 - 2t)$. Khi thắng, khối hạ thấp dần và các mặt bị cắt tại $z = 0$ (`clip_floor`) để không hiển thị phần khối chìm dưới sàn.

Hoạt ảnh rơi (`fall_motion.py`) `GameRules` chỉ trả `None` cho hành động khiến khối rơi, không giữ lại trạng thái trung gian để vẽ. `prepare_fall()` tính lại cục bộ, chỉ phục vụ hiển thị: áp dụng phần xoay của hành động (`rules._roll()`) để suy ra vị trí "định đáp", xác định vị trí còn được đỡ (`supported`) và ô dễ vỡ bị vỡ (`broken`); nếu còn đúng một điểm tựa, cạnh lật được xác định làm trục xoay. `fall_vertex()` dựng chuyển động hai giai đoạn: lật quanh cạnh đỡ trong 0.36s đầu, sau đó rơi tự do với gia tốc không đổi ($z \mathrel{-}= 2.5t + 15t^2$), khối mờ dần ở cuối hoạt ảnh.

Hoạt ảnh sao (`draw_victory()`) Độc lập với khối 3D: ba ngôi sao bật ra lệch nhau 0.12s mỗi sao, phóng to theo hàm *ease-out-back*

$$f(t) = 1 + c_3(t-1)^3 + c_1(t-1)^2, \qquad c_1 = 1.70158,\ c_3 = c_1 + 1,$$

tạo hiệu ứng vượt quá kích thước đích rồi co lại, khác với *smoothstep* êm dùng cho khối gỗ. Số sao đạt được tô vàng, số còn lại tô xám.

4.4. Luồng xử lý và tiến trình chơi

Vòng lặp chính trong `App.run()` lần lượt: xử lý sự kiện bàn phím/chuột, cập nhật tiến trình tìm kiếm đang chạy nếu có, tự động phát bước kế tiếp khi đang ở chế độ *Replay*, rồi vẽ lại khung hình. Phím mũi tên/WASD ứng với bốn hướng lăn trên bàn isometric (không trùng trục màn hình do góc chiếu), `Space` đổi khối con khi đã tách, `R` khởi động lại màn, `Esc` hủy tìm kiếm hoặc đóng panel đang mở theo thứ tự ưu tiên. Mỗi di chuyển hợp lệ hoặc không hợp lệ đều tạo một bản ghi `animation` dùng chung bởi `Renderer` và `wood.py` để phát đúng hoạt ảnh tương ứng.

Khi `GameController` báo trạng thái `"won"` ở chế độ chơi thủ công, `App.check_vict` đợi hoạt ảnh kết thúc rồi gọi `Progress.complete(map_name, moves)`: số sao được tính bằng `stars_for()` so với số bước mục tiêu (`TARGETS`), bản ghi tốt nhất được giữ lại và ghi xuống `.save/progress.json` theo mẫu ghi-tạm-rồi-đổi-tên (`.tmp` rồi `replace()`) để tránh hỏng file nếu chương trình bị tắt giữa lúc ghi. `unlocked(level)` yêu cầu mọi màn có số thứ tự nhỏ hơn đã hoàn thành, tức mở khóa tuần tự theo chiến dịch. Khi trạng thái `"lost"`, `App.check_defeat()` tự khởi động lại màn sau khi hoạt ảnh rơi phát xong, không cần người chơi nhấn `R`.

Chế độ *Solve by algorithm* dùng lại đúng `GameController` và `Renderer` nói trên nhưng không gọi `check_victory()`/`Progress.complete()` trong lúc phát lại, để kết quả giải bằng thuật toán không ghi sao hay mở khóa chiến dịch thay người chơi.

5. Tìm kiếm theo chiều sâu (DFS)

5.1. Mô hình trạng thái và chi phí

DFS tìm một chuỗi hành động hợp lệ đưa khối gỗ đến đích bằng cách ưu tiên duyệt theo chiều sâu. Mỗi trạng thái là một `GameState` gồm vị trí, hướng khối, khối gỗ đang điều khiển và tập cầu đang mở.

Các hành động gồm `UP`, `DOWN`, `LEFT`, `RIGHT` và `SWITCH`. Trạng thái kế tiếp được sinh bằng `GameRules.apply_action()`, bảo đảm xử lý thống nhất ô dễ vỡ,

công tắc, cầu và tách/ghép khối. Mọi hành động có chi phí bằng 1, kể cả đổi khối con. Với đường đi gồm k hành động:

$$g(n) = \sum_{i=1}^{k} c_i = k.$$

DFS lưu chi phí đường đi nhưng không dùng chi phí để chọn node mở rộng. Vì vậy thuật toán DFS không đảm bảo tìm được đường đi tối ưu nhất.

5.2. Cấu trúc dữ liệu và quá trình tìm kiếm

Lớp `DFSSolver` kế thừa `BaseSolver`, sử dụng ngăn xếp LIFO và tập `visited` chứa toàn bộ `GameState` để chống lặp. Trạng thái được đánh dấu khi node được lấy khỏi ngăn xếp, node có trạng thái đã được thăm sẽ được bỏ qua.

Các node con được thêm theo thứ tự ngược để ưu tiên xử lý `UP` → `DOWN` → `LEFT` → `RIGHT` → `SWITCH`. Khi gặp đích, thuật toán truy vết các trường `parent` và `action` của `SearchNode` để trả lời giải.

5.3. Tính chất và giới hạn

Với trạng thái hữu hạn, DFS có chống lặp sẽ tìm được lời giải nếu tồn tại và không bị dừng bởi giới hạn tài nguyên. Tuy nhiên DFS không đảm bảo tìm được lời giải tối ưu nhất. Kết quả còn phụ thuộc vào thứ tự duyệt các nhánh.

Gọi V và E là số trạng thái và cạnh trong đồ thị có thể đi tới. Nếu xem chi phí sinh mỗi trạng thái kế tiếp và tra bảng băm là hằng số, độ phức tạp thời gian là $O(V+E)$. Bộ nhớ phụ của cách cài đặt này cũng là $O(V+E)$ do lưu tập `visited` và có thể giữ nhiều node trùng trạng thái trong ngăn xếp. Mỗi trạng thái Bloxorz có tối đa năm hành động nên $E \leq 5V$.

5.4. Giao diện và kiểm thử

Lời gọi `DFSSolver.solve(initial_state)` trả một `SearchResult`, gồm trạng thái kết quả, chuỗi hành động, số node mở rộng, thời gian tìm kiếm và bộ nhớ đỉnh Python. Node đích và node trùng bị bỏ qua không được tính là node mở rộng. Khi chạy qua `Benchmark`, vượt giới hạn được báo là `limit`, không phải `no_solution`.

Các kiểm thử trong `tests/test_frontend.py` và `tests/test_benchmark.py` replay lời giải để xác nhận mọi hành động hợp lệ, trạng thái cuối đạt đích và đầu vào

Thuật toán 1 Thuật toán DFS

Đầu vào: Trạng thái đầu s_0, bộ luật sinh trạng thái kế tiếp
Đầu ra: Chuỗi hành động hợp lệ hoặc không có lời giải

$S \leftarrow$ ngăn xếp chứa node gốc của s_0
$visited \leftarrow \varnothing$
while S chưa rỗng **do**
    Lấy node n khỏi đỉnh S
    if $state(n) \in visited$ **then**
        Bỏ qua n và tiếp tục vòng lặp
    else
        Thêm $state(n)$ vào $visited$
        if $state(n)$ là đích **then**
            return chuỗi hành động truy vết từ n
        end if
        $children \leftarrow$ các successor hợp lệ của $state(n)$
        for mỗi (a, s', c) trong $children$ theo thứ tự đảo ngược **do**
            if $s' \notin visited$ **then**
                Tạo node con với trạng thái s', cha n, hành động a
                Gán độ sâu $depth(n)+1$ và chi phí $g(n)+c$
                Đưa node con vào S
            end if
        end for
    end if
end while
return không có lời giải

không bị thay đổi. Các trường hợp bao gồm màn cơ bản, cầu/công tắc, ô dễ vỡ và tách khối. Độ dài lời giải DFS không được yêu cầu bằng BFS vì DFS không bảo đảm tính tối ưu.

6. Tìm kiếm theo chi phí đồng nhất (UCS)

6.1. Mô hình trạng thái và chi phí

Thuật toán UCS (Uniform-Cost Search) tìm lời giải có tổng chi phí nhỏ nhất. Trong chương trình, mỗi đỉnh của đồ thị là một `GameState`, gồm trạng thái khối và tập cầu đang mở. Trạng thái khối thể hiện vị trí, hướng đặt, trạng thái tách đôi và khối con đang được điều khiển. Vì vậy, hai trạng thái cùng vị trí nhưng khác cấu hình cầu hoặc khối đang hoạt động vẫn có thể có các bước đi hợp lệ khác nhau.

Các cạnh được sinh qua `GameRules.apply_action()`, dùng chung luật chơi của nhóm. Tập hành động gồm lên, xuống, trái, phải và đổi khối con (`SWITCH`). Chỉ những hành động tạo được trạng thái hợp lệ mới trở thành cạnh. Trong phiên bản này, mọi hành động có chi phí bằng 1, kể cả đổi khối con. Với đường đi gồm k hành động:

$$g(n) = \sum_{i=1}^{k} c_i = k.$$

Do đó, tối thiểu hóa chi phí cũng là tối thiểu hóa số hành động.

6.2. Cấu trúc dữ liệu và quá trình tìm kiếm

Lớp `UCSSolver` kế thừa `BaseSolver`, dùng min-heap của Python để lấy node có chi phí nhỏ nhất. Mỗi phần tử heap có dạng $(g, \text{thứ tự thêm}, \text{node})$. Bộ đếm thứ tự giúp xử lý trường hợp đồng chi phí mà không phải so sánh trực tiếp các đối tượng trạng thái.

Bảng `best_cost` ánh xạ toàn bộ trạng thái tới chi phí thấp nhất đã biết. Khi tìm được đường đi tốt hơn, thuật toán cập nhật bảng và thêm một phần tử mới vào heap. Phần tử cũ vẫn có thể còn trong heap; khi lấy ra, nó được bỏ qua nếu chi phí không còn bằng giá trị tốt nhất. Thuật toán chỉ kiểm tra đích sau bước loại phần tử cũ này, khi node được lấy khỏi heap. Lời giải được truy vết qua các trường `parent` và `action` của `SearchNode` [**?**].

Thuật toán 2 UCS trên trạng thái Bloxorz

Đầu vào: Trạng thái đầu s_0, bộ luật sinh trạng thái kế tiếp
Đầu ra: Chuỗi hành động tối ưu hoặc không có lời giải

$Q \leftarrow$ min-heap chứa node gốc với chi phí 0
$best[s_0] \leftarrow 0$; các giá trị khác mặc định $+\infty$
while Q chưa rỗng **do**
    Lấy node n có chi phí nhỏ nhất khỏi Q
    if $g(n) \neq best[state(n)]$ **then**
        Bỏ qua n và tiếp tục vòng lặp
    else
        if $state(n)$ là đích **then**
            return chuỗi hành động truy vết từ n
        end if
        for mỗi (a, s', c) hợp lệ từ $state(n)$ **do**
            $g' \leftarrow g(n) + c$
            if $g' < best[s']$ **then**
                $best[s'] \leftarrow g'$
                Thêm node con (s', n, a, g') vào Q
            end if
        end for
    end if
end while
return không có lời giải

6.3. Tính chất và giới hạn

Với số trạng thái hữu hạn và chi phí cạnh dương, khi lấy node hợp lệ có chi phí nhỏ nhất ra khỏi heap, không thể có đường đi chưa xử lý nào tới node đó với chi phí thấp hơn: đường đi như vậy phải đi qua một node có chi phí nhỏ hơn, vốn đã được ưu tiên xử lý trước. Vì vậy, khi node đích được lấy ra, chi phí lời giải là tối ưu. Nếu heap rỗng, toàn bộ phần đồ thị có thể đi tới đã được duyệt mà không tìm thấy đích.

Gọi V và E là số trạng thái và cạnh trong đồ thị có thể đi tới. Nếu xem chi phí sinh trạng thái và tra bảng băm là hằng số, cách cài đặt heap cho phép giữ phần tử cũ có thời gian $O((V+E)\log(V+E))$ và bộ nhớ phụ $O(V+E)$ trong trường hợp xấu nhất. Các giới hạn thực tế của chương trình có thể dừng tìm kiếm trước khi thuật toán kết luận; trường hợp này được báo là vượt giới hạn, không phải vô nghiệm.

Vì mọi hành động hiện có cùng chi phí, BFS và UCS đều tối thiểu hóa số bước. UCS không được kỳ vọng luôn nhanh hơn BFS; heap còn có chi phí quản lý riêng. Lợi ích của cơ chế cập nhật chi phí thể hiện khi mở rộng bài toán sang các cạnh có trọng số khác nhau. Trường hợp này đã được kiểm thử bằng đồ thị tổng hợp, không thay đổi hàm chi phí của game.

7. Thuật toán A*

7.1. Mô hình hoá bài toán

Nhóm chúng em áp dụng kiến thức môn Cơ Sở Trí Tuệ Nhân Tạo vào để tiến hành mô hình hoá bài toán trước khi cài đặt thuật toán, cụ thể như sau:

- **State:** Mỗi trạng thái được biểu diễn bởi đối tượng `GameState`, gồm trạng thái khối và tập các cầu đang mở. Có thể ký hiệu trạng thái dưới dạng

$$s = (P, o, a, B),$$

trong đó P là bộ tọa độ các ô mà khối chiếm giữ; o là hướng của khối, nhận một trong bốn giá trị `STANDING` (đứng), `HORIZONTAL` (nằm theo trục x), `VERTICAL` (nằm theo trục y) hoặc `SPLIT` (tách thành hai khối con); a là chỉ số khối con đang được điều khiển khi khối bị tách; B là tập mã định danh của các cầu đang mở, được lưu bằng `frozenset`.

Khi khối đứng, P có một tọa độ; khi khối nằm hoặc bị tách, P có hai tọa độ. Các tọa độ được sắp xếp để chuẩn hóa biểu diễn; khi khối bị tách, chỉ số a được điều chỉnh theo thứ tự này để giữ đúng khối con đang được chọn. Trạng thái ban đầu s_0 được lấy từ `board.game_state`. Hai trạng thái có cùng vị trí và hướng nhưng khác tập cầu đang mở hoặc khác khối con đang được chọn vẫn là hai trạng thái khác nhau.

- **Goal:** Trạng thái đích là một trạng thái hợp lệ mà khối nguyên vẹn đang đứng trên ô có kiểu `Goal`. Điều kiện này được kiểm tra bởi `GameRules.is_goal`. Khối nằm ngang qua ô đích hoặc một khối con nằm trên ô đích chưa thỏa mãn điều kiện thắng. Mục tiêu của thuật toán là tìm chuỗi hành động hợp lệ từ s_0 đến một trạng thái đích với tổng chi phí nhỏ nhất.

- **Actions:** Tập hành động gồm `UP`, `DOWN`, `LEFT`, `RIGHT` và `SWITCH`. Bốn hành động đầu làm khối nguyên vẹn lăn theo hướng tương ứng; nếu khối đã bị tách, chúng chỉ di chuyển khối con đang được chọn một ô. Hành động `SWITCH` chỉ hợp lệ khi khối bị tách và dùng để đổi khối con đang được điều khiển, không làm thay đổi vị trí. Tại mỗi trạng thái, `get_valid_actions` chỉ trả về các hành động hợp lệ. Mỗi hành động, kể cả `SWITCH`, có chi phí bằng 1, nên chi phí đường đi $g(s)$ chính là số hành động đã thực hiện.

- **Transition Model:** Hàm `apply_action` của `GameRules` xác định trạng thái kế tiếp:

$$T(s,u) = s', \qquad u \in A(s),$$

với $A(s)$ là tập hành động hợp lệ tại s. Đối với hành động di chuyển, hàm tính vị trí và hướng mới theo tư thế hiện tại, sau đó kiểm tra hình dạng khối và khả năng nâng đỡ của các ô bên dưới. Di chuyển ra ngoài bàn cờ, đi lên cầu đang đóng hoặc đứng trên ô dễ vỡ đều không hợp lệ.

Sau khi khối đáp xuống, các công tắc được kích hoạt sẽ được xử lý theo thứ tự mã định danh để cập nhật cầu bằng hiệu ứng `TOGGLE` hoặc `SET`. Công tắc tách khối còn đưa hai khối con đến các vị trí được cấu hình. Trạng thái sau hiệu ứng được kiểm tra lại; nếu khối trước hành động đã ở dạng tách và hai khối con sau di chuyển nằm kề nhau theo cạnh, chúng được ghép thành một khối nằm. Riêng `SWITCH` chỉ đổi chỉ số khối con đang được chọn, không kích hoạt công tắc hay ghép khối. Hàm trả về trạng thái mới khi hợp lệ và `None` nếu hành động không hợp lệ, đồng thời không thay đổi trạng thái đầu vào.

- **Heuristic Function:** Thuật toán đánh giá mỗi nút bằng

$$f(s) = g(s) + h(s).$$

Trong cài đặt, $h(s)$ được tra từ `heuristic_table`, được xây dựng một lần khi khởi tạo bộ giải bằng hàm `_build_heuristic_table`. Nhóm sử dụng khoảng cách ngắn nhất trên một bài toán nới lỏng: tất cả cầu luôn mở và các hiệu ứng thay đổi cầu của công tắc được loại bỏ. Các quy tắc lăn khối, ô dễ vỡ, tách khối, dịch chuyển khối con, đổi khối con và ghép khối vẫn được giữ nguyên.

Bộ giải liệt kê các cấu hình khối đứng, nằm và tách trên các vị trí của bàn cờ, bao gồm cả vị trí cầu. Với cấu hình tách, cả hai lựa chọn khối con đang được điều khiển đều được xét. Từ các cấu hình hợp lệ, bộ giải sinh các chuyển trạng thái và lưu danh sách tiền nhiệm của từng cấu hình. Sau đó, BFS ngược được thực hiện đồng thời từ tất cả cấu hình đứng trên ô đích để tính

$$h(s) = d_{\text{relaxed}}\big(\text{block}(s), \mathcal{G}_{\text{relaxed}}\big),$$

trong đó d_{relaxed} là số hành động ít nhất trên đồ thị nới lỏng và $\mathcal{G}_{\text{relaxed}}$ là tập cấu hình đích. Việc lưu tiền nhiệm giúp BFS đi ngược đúng các cạnh có hướng, kể cả khi công tắc gây dịch chuyển khối. Khóa tra cứu chỉ là `state.block`, không bao gồm tập cầu đang mở. Cấu hình đích có giá trị 0; cấu hình không có đường đến đích trong bài toán nới lỏng nhận giá trị $+\infty$.

Do việc mở sẵn mọi cầu và bỏ hiệu ứng thay đổi cầu không loại bỏ đường đi hợp lệ nào của bài toán gốc khi chỉ xét cấu hình khối, heuristic không đánh giá quá cao chi phí còn lại, tức là $h(s) \le h^*(s)$. Với mỗi chuyển trạng thái hợp lệ $s \to s'$, ta còn có $h(s) \le 1 + h(s')$. Vì vậy, heuristic có tính chấp nhận được và nhất quán, giúp A* tìm lời giải có số hành động ít nhất.

- **Avoid revisiting states:** Bộ giải sử dụng từ điển `g_score` để lưu chi phí nhỏ nhất đã tìm được đến từng `GameState`. Ban đầu, $g_score[s_0] = 0$. Khi sinh trạng thái kế tiếp s' từ s, thuật toán tính $g_{\text{new}} = g(s) + 1$. Nếu $g_{\text{new}} \ge g_score[s']$, trạng thái này bị bỏ qua; nếu nhỏ hơn, thuật toán cập nhật chi phí và đưa nút mới vào hàng đợi ưu tiên. Trạng thái chưa có trong từ điển được xem là có chi phí $+\infty$.

Hàng đợi `PriorityQueue` lưu bộ ba $(f(s), \texttt{counter}, \texttt{node})$, ưu tiên nút có $f(s)$ nhỏ nhất; bộ đếm tăng dần phân xử các nút có cùng giá trị f theo thứ tự đưa

vào. Khi lấy một nút ra, nếu chi phí của nút lớn hơn giá trị hiện có trong `g_score`, nút đó bị bỏ qua vì đã có đường đi tốt hơn. Cơ chế này hạn chế duyệt lặp nhưng vẫn cho phép xét lại một trạng thái khi tìm được đường đi có chi phí thấp hơn. Khi lấy được nút đích, bộ giải truy vết các liên kết cha để trả về chuỗi hành động; nếu hàng đợi rỗng mà chưa gặp đích, bộ giải trả về `None`.

7.2. Về hàm Heuristic

Ban đầu, em cân nhắc thiết kế hàm heuristic dựa trên khoảng cách Manhattan. Tuy nhiên, khoảng cách giữa các ô chưa phản ánh đầy đủ số hành động trong Bloxorz: một lần lăn có thể làm tọa độ khối thay đổi nhiều hơn một ô, tư thế khối ảnh hưởng đến cách di chuyển, và công tắc tách khối có thể đưa các khối con đến vị trí khác trên bản đồ. Vì vậy, nếu dùng trực tiếp khoảng cách Manhattan làm số bước còn lại thì chưa thể bảo đảm tính chấp nhận được. Sau khi trao đổi, nhóm em lựa chọn heuristic dựa trên khoảng cách ngắn nhất trong một bài toán nới lỏng, được tính trước bằng **BFS ngược đa nguồn**.

7.2.1. Xây dựng bài toán nới lỏng

Bước đầu tiên là tạo một bản sao của bàn cờ, mở sẵn tất cả các cầu và loại bỏ các hiệu ứng thay đổi cầu của công tắc. Trong mã nguồn, các công tắc được sao chép với thuộc tính `effects` rỗng; tập `open_bridges` chứa mã định danh của toàn bộ cầu. Nhờ đó, việc đi qua một cầu không còn phụ thuộc vào lịch sử kích hoạt công tắc.

Việc nới lỏng chỉ áp dụng cho cơ chế đóng, mở cầu. Các ô dễ vỡ vẫn không cho phép khối đứng lên; công tắc tách khối vẫn tách và dịch chuyển hai khối con đến vị trí đã cấu hình. Các quy tắc lăn, di chuyển khối con, đổi khối con đang được điều khiển và ghép khối vẫn được thực hiện bởi `GameRules`. Giữ lại các quy tắc này giúp giá trị ước lượng phản ánh tư thế và khả năng di chuyển của khối.

Vì trạng thái cầu đã được cố định, mỗi đỉnh trong đồ thị nới lỏng chỉ cần lưu một `BlockState`, gồm vị trí, hướng và chỉ số khối con đang được chọn. Gọi đồ thị này là $G_R = (V_R, E_R)$ và tập cấu hình đứng trên ô đích là $\mathcal{G}_R$. Với trạng thái thực s, heuristic được định nghĩa bởi

$$h(s) = \min_{q_g \in \mathcal{G}_R} d_{G_R}\big(\text{block}(s), q_g\big),$$

trong đó d_{G_R} là số hành động ít nhất giữa hai cấu hình trên đồ thị nới lỏng. Đây là khoảng cách chính xác trong bài toán nới lỏng và là ước lượng chi phí còn lại trong bài toán gốc.

7.2.2. Xây dựng bảng khoảng cách bằng BFS ngược

Hàm `_build_heuristic_table` thực hiện các bước sau:

1. Thu thập tập tọa độ của các ô và các cầu. Tại mỗi tọa độ, sinh cấu hình đứng; nếu có ô kề theo trục x hoặc y, sinh thêm cấu hình nằm tương ứng. Với mỗi cặp tọa độ phân biệt, sinh cấu hình tách cho cả hai lựa chọn khối con đang được điều khiển.

2. Gắn từng cấu hình với tập cầu luôn mở và kiểm tra bằng `is_valid`. Với mỗi cấu hình hợp lệ q, thử cả năm hành động bằng `apply_action`. Nếu một hành động đưa q đến q', thêm q vào danh sách tiền nhiệm của q'. Như vậy, cạnh $q \to q'$ của đồ thị thuận được lưu để có thể duyệt ngược từ q' về q.

3. Gán khoảng cách 0 cho tất cả cấu hình đứng trên ô đích và đưa chúng vào hàng đợi `deque`. Đây là các nguồn khởi đầu của BFS trên đồ thị đảo cạnh.

4. Lần lượt lấy cấu hình q' ở đầu hàng đợi. Với mỗi tiền nhiệm q chưa có khoảng cách, gán

$$\text{distance}[q] = \text{distance}[q'] + 1$$

rồi thêm q vào cuối hàng đợi. Mỗi cấu hình chỉ được gán khoảng cách ở lần đầu tiên được phát hiện. Khi hàng đợi rỗng, trả về từ điển khoảng cách làm bảng heuristic.

Mọi hành động đều có chi phí 1, kể cả hành động đổi khối con, nên BFS duyệt theo từng lớp khoảng cách và lần phát hiện đầu tiên cho khoảng cách ngắn nhất. DFS không bảo đảm tính chất này. Việc sử dụng danh sách tiền nhiệm cũng cần thiết vì các chuyển trạng thái có thể có hướng: không thể mặc định rằng thực hiện hành động ngược sẽ đảo được hiệu ứng tách khối hoặc dịch chuyển do công tắc gây ra.

Trong quá trình chạy A*, hàm `_heuristic_function` chỉ tra bảng bằng khóa `state.block`. Hai trạng thái thực có cùng cấu hình khối nhưng khác tập cầu đang mở sẽ nhận cùng giá trị heuristic, mặc dù chúng vẫn được A* phân biệt khi lưu `g_score`. Cấu hình không có trong bảng nhận giá trị `math.inf`, tức là không có đường đến đích ngay cả trong bài toán nới lỏng. Giá trị này không làm thuật toán kết thúc sớm: cài đặt hiện tại vẫn đưa nút vào hàng đợi ưu tiên với giá trị f tương ứng.

7.2.3. Admissible and Consitent

Xét một lời giải của bài toán gốc có k hành động. Khi mở sẵn tất cả cầu và bỏ các hiệu ứng thay đổi cầu, chuỗi cấu hình khối của lời giải đó vẫn tạo thành một đường đi hợp lệ

đến đích trong đồ thị nới lỏng. Do BFS trả về khoảng cách ngắn nhất trên đồ thị này, ta có

$$h(s) \leq k.$$

Chọn $k = h^*(s)$, với $h^*(s)$ là chi phí lời giải tối ưu từ s, suy ra $h(s) \leq h^*(s)$. Vì vậy, heuristic có tính chấp nhận được. Đặc biệt, trạng thái đích có $h(s) = 0$; nếu $h(s) = +\infty$ thì trạng thái thực cũng không thể có lời giải.

Đối với tính nhất quán, xét một chuyển trạng thái hợp lệ $s \to s'$. Chuyển động tương ứng giữa hai cấu hình khối cũng tồn tại trong đồ thị nới lỏng với chi phí 1. Đi từ cấu hình của s sang cấu hình của s', rồi theo đường ngắn nhất đến đích từ đó, là một phương án đến đích. Khoảng cách ngắn nhất từ cấu hình của s không thể lớn hơn chi phí của phương án này, nên

$$h(s) \leq 1 + h(s').$$

Do đó, heuristic có tính nhất quán. Kết hợp với chi phí hành động bằng 1 và cơ chế cập nhật chi phí tốt nhất trong `g_score`, A* trả về lời giải có số hành động ít nhất khi tìm được đích.

7.2.4. Chi phí tính toán và giới hạn

Bảng heuristic được xây dựng một lần khi khởi tạo một đối tượng `AStarSolver` và được dùng lại trong quá trình tìm kiếm của đối tượng đó. Mỗi lần tra cứu có chi phí trung bình $O(1)$ nhờ sử dụng từ điển, thay vì phải chạy một phép tìm đường mới cho từng nút.

Đổi lại, bước tiền xử lý phải liệt kê cả các cấu hình tách khối. Nếu tập tọa độ có n phần tử, riêng phần này có $2\binom{n}{2} = n(n-1)$ cấu hình ứng viên, nên tổng số cấu hình ứng viên tăng theo $O(n^2)$, kể cả khi bản đồ không có công tắc tách khối. Sau khi có đồ thị, BFS ngược cần thời gian $O(|V_R| + |E_R|)$; việc lưu đồ thị tiền nhiệm và bảng khoảng cách cần bộ nhớ $O(|V_R| + |E_R|)$. Chi phí dựng đồ thị còn bao gồm việc kiểm tra luật và tra cứu ô trên bàn cờ, nên không thể đồng nhất toàn bộ thời gian tiền xử lý với riêng chi phí BFS.

Heuristic vẫn có thể nhỏ hơn đáng kể so với chi phí thực tế nếu lời giải phải đi vòng để mở cầu. Đây là phần thông tin đã được bỏ qua khi nới lỏng bài toán. Các kiểm thử hiện có đối chiếu độ dài lời giải A* với BFS trên năm bản đồ `map01`, `map02`, `map04`, `map08` và `map09`; đồng thời kiểm tra bất đẳng thức nhất quán trên các chuyển trạng thái được BFS khảo sát và tính chấp nhận được dọc theo lời giải tối ưu tìm được.

8. Tích hợp thuật toán vào giao diện game

8.1. Phạm vi tích hợp

Trên nền giao diện Pygame và bộ luật chơi có sẵn của nhóm, phần tích hợp bổ sung chế độ *Solve by algorithm*. Người dùng chọn BFS, DFS, UCS hoặc A*, chạy tìm kiếm, xem kết quả đo, phát lại lời giải và xuất CSV. Các solver dùng chung `GameRules`; giao diện không cài đặt lại luật di chuyển, công tắc, cầu hoặc tách khối [**?**].

Luồng xử lý được tổ chức thành ba phần:

1. `FrontEnd/app.py` nhận lựa chọn, lưu trạng thái bắt đầu và tạo tiến trình tìm kiếm.
2. `BackEnd/search.py` chọn lớp solver và gọi `Benchmark` để đo quá trình khởi tạo, tìm kiếm.
3. Kết quả được trả về UI; `GameController` lưu chuỗi hành động và điều khiển replay từ đúng trạng thái đã bắt đầu tìm kiếm.

8.2. Chạy, dừng và phát lại lời giải

Tìm kiếm bắt đầu từ vị trí hiện tại, bao gồm trạng thái cầu và khối con đang hoạt động. Vì vậy, trước khi so sánh các thuật toán, cần khởi động lại cùng một màn để bảo đảm đầu vào giống nhau.

Solver chạy trong tiến trình riêng bằng cơ chế `spawn`, giúp vòng lặp giao diện tiếp tục xử lý sự kiện. Người dùng có thể hủy tìm kiếm; đổi màn, chơi lại, trở về chế độ thủ công hoặc đóng cửa sổ cũng thu hồi worker. UI có thêm thời hạn giám sát để dừng worker không phản hồi và không sử dụng kết quả cũ sau khi trạng thái chơi thay đổi.

Khi tìm được lời giải, *Replay/Pause* phát hoặc tạm dừng chuỗi hành động, còn *Step* thực hiện một hành động. Tốc độ phát lại điều chỉnh từ 1 đến 10 hành động/giây. Replay lại từ đầu khôi phục trạng thái xuất phát của lần tìm kiếm; thao tác di chuyển thủ công loại bỏ lời giải và số liệu cũ. Việc phát lại lời giải tự động không ghi sao hoặc mở khóa chiến dịch thủ công.

8.3. Hiển thị kết quả và xuất dữ liệu

Bảng benchmark hiển thị thời gian, đỉnh bộ nhớ Python, số node mở rộng và số hành động/chi phí lời giải. UI phân biệt bốn trạng thái: `solved`, `no_solution`, `limit` và `error`. Nếu worker không trả được bộ đếm hoặc số liệu bộ nhớ, giao diện hiển thị không có số liệu thay vì ghi nhận một phép đo bằng 0.

Nút *Export CSV* nối kết quả vào `output/search_results.csv`, kèm tên map, trạng thái bắt đầu và chuỗi hành động. File xuất từ UI dùng định dạng riêng; nó chưa đồng nhất với schema benchmark hàng loạt ở Mục 9. Do đó, không ghép hai loại CSV theo vị trí cột.

8.4. Tương thích sau khi tích hợp vào main

Khi merge nhánh UI vào `main`, các thay đổi ở `BaseSolver`, BFS và DFS cần được phối hợp. Phiên bản tích hợp giữ API `DFS.solve()` trả `SearchResult`, giữ `BFS._search()` trả node đích và số node mở rộng; lời gọi trực tiếp `solve()` của BFS, UCS và A* trả danh sách hành động hoặc `None`. Wrapper `BackEnd.search.solve()` chuẩn hóa cả bốn thuật toán thành cùng lớp `SearchResult` cho UI. DFS truyền ngoại lệ giới hạn ra wrapper để tránh báo nhầm việc dừng tìm kiếm thành lỗi.

9. Benchmark và trực quan hóa kết quả

9.1. Thiết kế phép đo

Bộ dữ liệu được xây dựng trong giai đoạn tích hợp UI gồm 20 maps, 4 thuật toán và 1 lần chạy cho mỗi cặp, tổng cộng 80 lượt. Mỗi lượt dùng trạng thái khởi tạo của map và cùng bộ luật chơi. Mã nguồn được đo là commit `29f2323`; môi trường được ghi nhận là Python 3.14.7, macOS 26.6.2 trên ARM64. Dữ liệu và phương pháp được lưu trong `benchmarks/benchmark_20_maps.csv` và tài liệu schema đi kèm [**?**].

Đây là kết quả của phiên bản lịch sử, không phải số đo mới của `main` hiện tại. Repository hiện có thêm 15 maps và dữ liệu riêng; phần phân tích này giữ phạm vi 20 maps ban đầu, không gộp các đợt đo có phiên bản code khác nhau. Đặc biệt, cách cài đặt DFS đã thay đổi khi tích hợp với `main`, nên không dùng CSV cũ để khẳng định hiệu năng của DFS hiện tại.

Các lượt chạy được thực hiện tuần tự theo map01 đến map20, trong mỗi map theo

thứ tự BFS, DFS, UCS, A*. Mỗi lượt tạo bàn cờ và tiến trình `spawn` mới. Giới hạn nội bộ là 30 giây và 150.000 node mở rộng; tiến trình cha dùng deadline 35 giây, kể cả khởi động worker.

`Benchmark` đo cả kiểm tra đầu vào, khởi tạo solver và tìm kiếm, bao gồm dựng bảng heuristic của A*. Phép đo không bao gồm đọc map, khởi động tiến trình, vẽ UI, truyền kết quả hoặc replay kiểm chứng. Các đại lượng được sử dụng như sau:

- Thời gian: chênh lệch `perf_counter`, đơn vị giây, có overhead của cơ chế theo dõi bộ nhớ.
- Bộ nhớ: đỉnh cấp phát Python do `tracemalloc` theo dõi, không phải tổng RAM/RSS của tiến trình; khi vẽ đổi byte sang MiB bằng cách chia cho 2^{20}.
- Node mở rộng: số node thực sự sinh successors; không tính node đích, heap entry cũ hoặc tiền xử lý heuristic.
- Chất lượng lời giải: số hành động và tổng chi phí. Hai giá trị bằng nhau vì mọi hành động đều có chi phí 1.

9.2. Schema CSV và kiểm chứng dữ liệu

Schema phiên bản `1.0.0` có 21 cột cố định, mỗi dòng biểu diễn một lượt chạy. Khóa duy nhất là `(run_id, map_id, algorithm, trial)`. Các cột được nhóm theo vai trò để tiện trình bày trong Bảng 4; tên, thứ tự và kiểu đầy đủ nằm trong `docs/benchmark_csv_schema.md`.

Bảng 4: Các nhóm thông tin trong schema benchmark

Nhóm	Thông tin lưu trữ
Nhận diện lượt đo	Phiên bản schema, run ID, thời điểm, map, thuật toán, số thứ tự lần chạy.
Nguồn và môi trường	Commit Git, phiên bản Python, nền tảng, SHA-256 của file maps.
Kết quả và giới hạn	Trạng thái, thời gian, bộ nhớ, số node, giới hạn thời gian và giới hạn node.
Lời giải và kiểm chứng	Độ dài, chi phí, chuỗi hành động, cờ replay hợp lệ và thông báo.

Ô trống của trường nullable biểu diễn dữ liệu không có, không thay bằng 0. Chỉ khi `status=solved` mới có độ dài và chi phí lời giải. Trạng thái `limit` không chứng minh map vô nghiệm.

Sau mỗi lượt giải, chuỗi hành động được replay độc lập ngoài khoảng đo bằng `apply_action()` và kiểm tra kết thúc bằng `is_goal()`. Bộ dữ liệu có đủ 80 khóa

duy nhất, 80 lượt `solved` và 80 lời giải replay hợp lệ. Notebook kiểm tra schema, tính đầy đủ, độ dài chuỗi và các cờ kiểm chứng đã lưu; notebook không chạy lại solver hay tự replay lại luật chơi.

9.3. Kết quả trên 20 maps

Bảng 5 tổng hợp trung vị của từng đại lượng trên 20 maps. Đây là trung vị giữa các map, không phải trung vị của nhiều lần lặp trên cùng một map. Giá trị node hoặc bước có phần thập phân xuất hiện do lấy trung vị của số lượng mẫu chẵn.

Bảng 5: Trung vị các số đo trong bộ dữ liệu 20 maps

Thuật toán	Giây	MiB	Node	Bước
BFS	0.0574	0.1561	301.0	45.0
DFS	0.0419	0.1136	207.5	90.5
UCS	0.0615	0.1329	301.0	45.0
A*	0.9438	2.0428	110.0	45.0

BFS, UCS và A* cho cùng độ dài lời giải trên cả 20 maps. BFS và UCS cũng có cùng số node mở rộng trên từng map trong bộ dữ liệu này, phù hợp với hàm chi phí đơn vị và quy tắc xử lý đồng chi phí. Quan sát này không có nghĩa hai thuật toán có cùng thời gian hoặc bộ nhớ.

DFS có lời giải dài bằng BFS trên 1/20 maps. Riêng map10, DFS dùng 1.539 hành động, trong khi BFS, UCS và A* dùng 58 hành động, tương đương khoảng 26,53 lần. Vì vậy, thời gian tìm kiếm nhỏ không đồng nghĩa lời giải có ít bước thực hiện.

A* mở rộng ít node hơn BFS trên cả 20 maps nhưng chỉ nhanh hơn trên 1/20 maps. Để so sánh tương đối, với đại lượng x và thuật toán a, notebook tính tỷ lệ trên từng map rồi mới lấy trung vị:

$$R_{a,x} = \text{median}_{m=1,\ldots,20} \left(\frac{x_{m,a}}{x_{m,\text{BFS}}} \right).$$

Theo cách tính này, A* có tỷ lệ node khoảng 0.43, thời gian 14.30 và bộ nhớ 13.89 so với BFS. Trung vị tỷ lệ không bằng tỷ lệ của hai trung vị trong Bảng 5. Kết quả cần được đọc cùng phạm vi đo: thời gian và bộ nhớ có tính dựng heuristic, còn node mở rộng chỉ tính giai đoạn tìm kiếm. Dữ liệu chưa tách thời gian tiền xử lý, nên chưa định lượng riêng phần chi phí này.

9.4. Lựa chọn biểu đồ

Notebook `visualization/benchmark_results.ipynb` và module `visualization/benchmark_plots.py` tạo bảy biểu đồ, xuất cả PNG và SVG [?]. Mỗi loại biểu đồ trả lời một câu hỏi khác nhau:

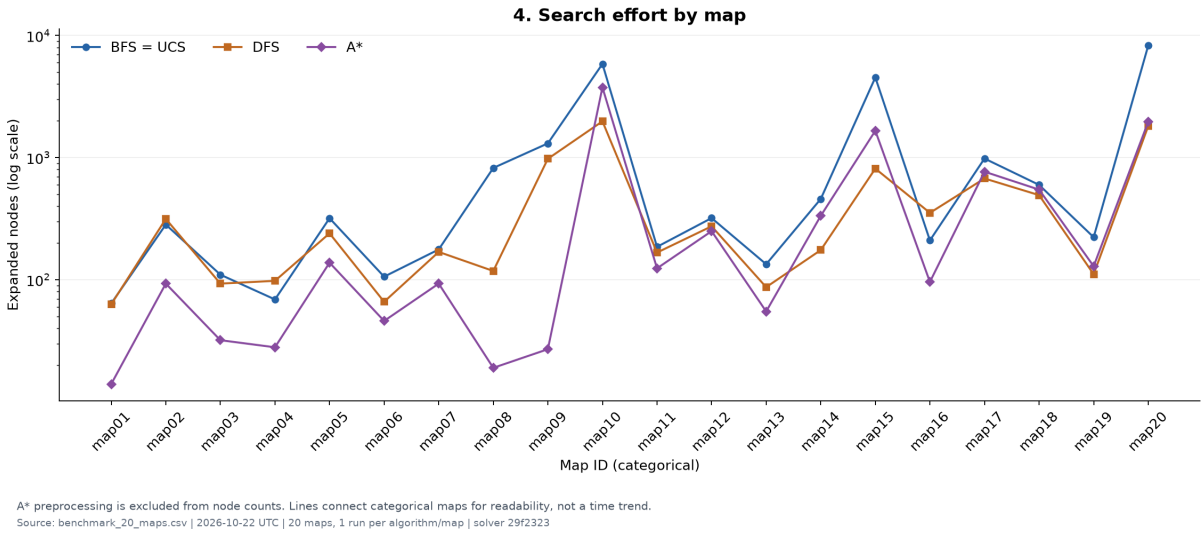
1. Heatmap thời gian: thuật toán nào tốn thời gian ở map nào.
2. Boxplot kèm các điểm thời gian: phân bố thời gian giữa các map.
3. Boxplot kèm các điểm bộ nhớ: phân bố đỉnh cấp phát Python.
4. Đồ thị node theo map: mức mở rộng không gian tìm kiếm.
5. Dumbbell độ dài lời giải: chênh lệch số bước DFS so với nhóm BFS/UCS/A* có cùng độ dài trong dữ liệu.
6. Scatter node–thời gian: quan hệ quan sát giữa số node và thời gian tổng, không suy ra quan hệ nhân quả.
7. Heatmap tỷ lệ so với BFS: so sánh các đại lượng trên cùng map trước khi tổng hợp, không gộp thành một điểm xếp hạng tùy ý.

Ba hình dưới đây minh họa số node, độ dài lời giải và tỷ lệ tổng hợp. Đồ thị có trục log được ghi rõ; chỉ số map là nhãn phân loại, không phải thời gian hoặc thước đo độ khó tăng dần.

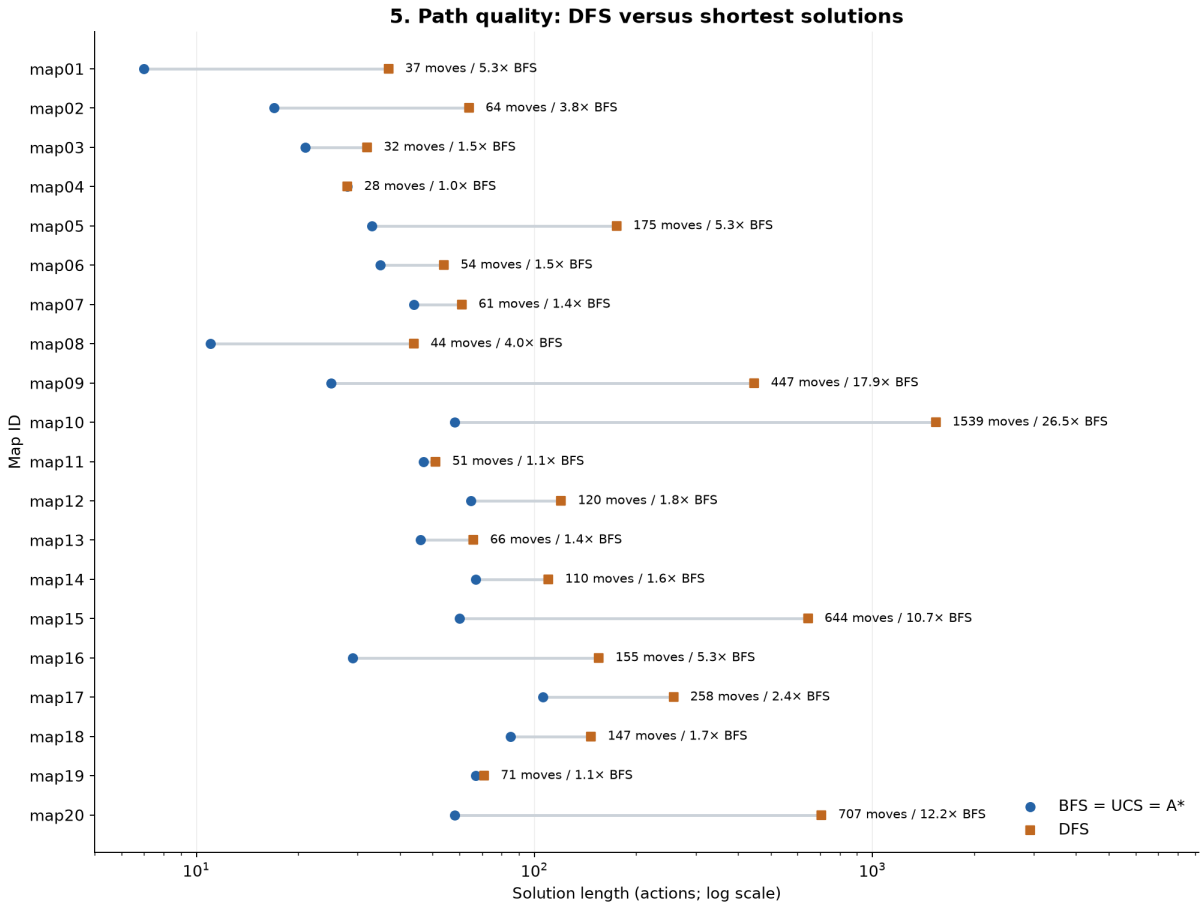
9.5. Hạn chế và hướng mở rộng

Mỗi cặp map–thuật toán chỉ có một lượt đo nên chưa thể đánh giá độ ổn định hoặc xây dựng khoảng tin cậy về thời gian. Chênh lệch nhỏ giữa BFS và UCS không đủ để kết luận ưu thế hiệu năng tổng quát. Boxplot hiện mô tả sự khác nhau giữa các maps, không mô tả nhiễu qua nhiều lần chạy cùng đầu vào.

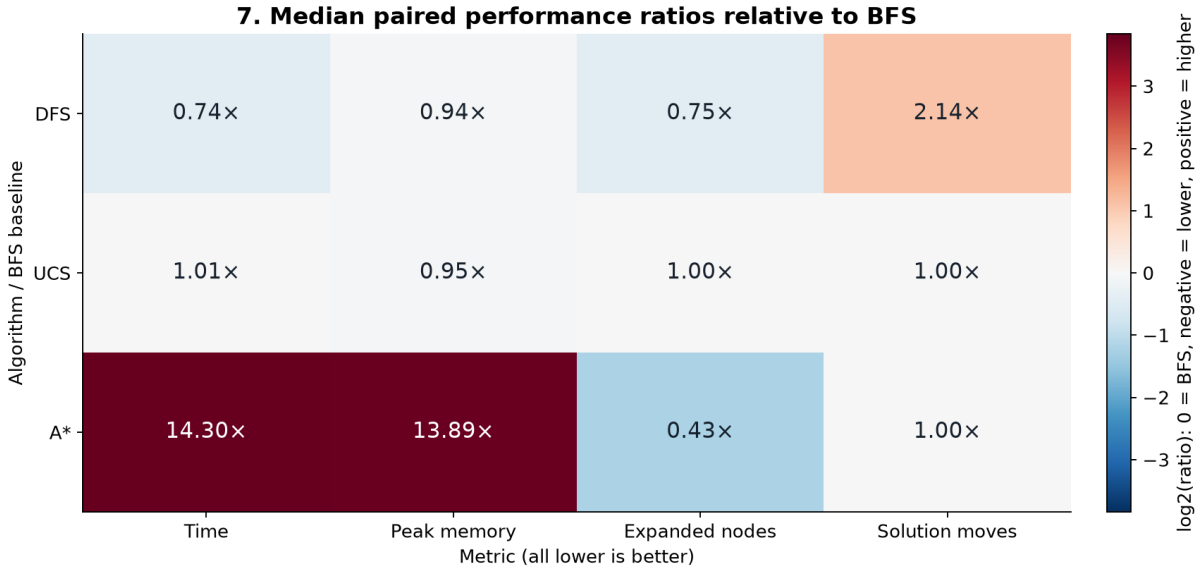
Các bước tiếp theo phù hợp là lặp lại phép đo trên cùng phiên bản code, lưu số thứ tự trial, thay đổi thứ tự chạy để giảm ảnh hưởng thứ tự, và tách thời gian dựng heuristic khỏi thời gian tìm kiếm khi cần phân tích sâu A*. Nếu đo thêm RSS hoặc thay đổi ý nghĩa các cột, cần bổ sung đại lượng hoặc nâng phiên bản schema thay vì ghi đè ý nghĩa của số liệu hiện có.



Hình 4: Node mở rộng trên 20 maps. Hai chuỗi BFS và UCS trùng nhau trong dữ liệu nên được gộp thành một đường. Nguồn: CSV tại commit solver `29f2323`.



Hình 5: So sánh độ dài lời giải DFS với BFS/UCS/A* trên 20 maps. Trục số hành động dùng thang log; hệ số thể hiện số bước DFS chia cho số bước BFS trên cùng map.



Hình 6: Trung vị tỷ lệ theo từng map so với BFS. Giá trị 1 là bằng BFS; thấp hơn 1 là nhỏ hơn ở đại lượng tương ứng. Màu dùng thang log cơ số 2 để biểu diễn tỷ lệ.

Tài liệu tham khảo